Exact Enumeration of Four-Window Permutations: OEIS A180207 through $n = 22$

Carlo Corti
Independent researcher
carlo.corti@outlook.com

November 2026

Abstract

OEIS A180207 counts ordered permutations of $0, \ldots, n-1$ that are consecutive sums of four entries of a nonnegative integer sequence of length $n+3$. We give an exact enumeration through $n = 22$, extending the historical 16-term prefix used in this study by six locally computed values, ending in $a(22) = 2055385215352$. The zero entry forces four consecutive zeros in the realization, makes that realization unique, and decomposes each permutation into two outward tails. A weighted subset dynamic program counts these tails and combines complementary label sets. A second complete count through 22 uses inclusion–exclusion on walks with repeated labels; an unrestricted-window Python method supplies an additional check at $1, \ldots, 12, 16, 17$. We prove a residue-minimum feasibility criterion, the counting identities, and bounds for their intermediate arithmetic. Reversal gives evenness for $n > 1$, while two end-extension injections give $a(n+1) \geq 2a(n)$.

2020 Mathematics Subject Classification. Primary 05A05; Secondary 05A15, 05A19.

Key words and phrases. permutation, consecutive sum, nonnegative realization, subset dynamic programming, inclusion–exclusion, OEIS A180207.

1 Introduction

Consider a permutation $p = (p_0, \ldots, p_{n-1})$ of $\{0, \ldots, n-1\}$. We ask whether there is an integer vector $x = (x_0, \ldots, x_{n+2})$ with nonnegative entries such that

$$p_i = x_i + x_{i+1} + x_{i+2} + x_{i+3} \qquad (0 \leq i < n). \tag{1}$$

Let $\mathcal{A}_n$ be the set of such permutations and let $a(n) = |\mathcal{A}_n|$, for $n \geq 1$, with the same index n and initial index 1 as OEIS A180207. The OEIS entry credits R. H. Hardin with author date 15 August 2010 [1]. The OEIS snapshot used for the historical baseline contains the values through $n = 16$, ending in 35154332. These are the historical baseline of the present study and retain Hardin's attribution; the entry's author date does not date the computation of each individual term.

Related OEIS entries replace the window length four by two, three, five, six and seven: A180205, A180206, A180208, A180209 and A180210, respectively [2]. These are variants of the definition; no exact transform from their counts to $a(n)$ is used here. The forward problem of counting distinct interval sums of a permutation is studied by Konieczny [3], while Higashitani and Kurimoto [4] study the minimum possible maximal cyclic window sum. Their input is a permutation, whereas here it is the output word of the window operation that must be a permutation. The methods below address this inverse nonnegative representation problem directly.

The permutations are counted as ordered words. Reversal, rotation, complementation and inversion are not identification rules, and the entries of x need not be distinct. The problem is an inverse representation problem: the window sums must be a permutation, whereas the realizing vector is only required to be nonnegative. For example, $(0,1,2)$ is realized by $(0,0,0,0,1,1)$, while $(1,0,2)$ is realized by $(1,0,0,0,0,2)$.

A direct scan of all $n!$ permutations becomes expensive even with a linear feasibility test. The useful feature here is the label zero. If $p_k = 0$, then $x_k = x_{k+1} = x_{k+2} = x_{k+3} = 0$. This determines the entire realization and separates its left and right parts. The enumeration can therefore be organized by the label sets on those parts, retaining just the last three realization entries as continuation state.

This paper proves the zero decomposition and its dynamic programming recurrence, and reports a complete C enumeration through 22 performed by the author. After that run, a separately implemented inclusion–exclusion method recounted all values through 22. A third method, which generates from unrestricted initial triples rather than from the zero separator, checks a smaller, explicitly stated domain. The six values additional to the historical baseline are given in Section 7. The comparison is with that baseline; no claim of worldwide priority or of the current remote OEIS endpoint is needed for the numerical results.

2 Realizations and a feasibility criterion

Subtracting consecutive equations in (1) gives

$$x_{i+4} - x_i = p_{i+1} - p_i \qquad (0 \le i \le n-2). \tag{2}$$

Thus the initial four entries determine every subsequent entry.

Lemma 2.1 (Uniqueness)**.** *Every $p \in \mathcal{A}_n$ has exactly one nonnegative integer realization.*

Proof. Let $p_k = 0$. Nonnegativity forces $x_k, \dots, x_{k+3}$ to be zero. These four consecutive positions include one representative of each residue class modulo four. Equation (2), propagated both forward and backward within each class, determines every other entry. Existence holds by the definition of $\mathcal{A}_n$, so the realization is unique. For $n = 1$ the sole window is zero and the same argument applies. □

There is also a direct test that does not require finding the zero position. For $r \in \{0,1,2,3\}$ put

$$K_r = \left\lfloor \frac{n+2-r}{4} \right\rfloor, \qquad D_r(0) = 0, \qquad D_r(k) = \sum_{j=0}^{k-1} (p_{r+4j+1} - p_{r+4j}) \quad (1 \le k \le K_r),$$

and define

$$m_r = -\min_{0 \le k \le K_r} D_r(k). \tag{3}$$

All differences appearing here have indices in the range of (2). In particular $K_r \ge 0$, and $m_r \ge 0$ because $D_r(0) = 0$.

Proposition 2.2 (Residue-minimum criterion)**.** *For any word $p \in \mathbb{Z}_{\ge 0}^n$, equation (1) has a nonnegative integer solution if and only if*

$$\sum_{r=0}^{3} m_r \le p_0. \tag{4}$$

If p is a permutation of $\{0, \dots, n-1\}$, feasibility forces equality in (4), and its unique realization is

$$x_{r+4k} = m_r + D_r(k) \qquad (0 \le r \le 3,\ 0 \le k \le K_r). \tag{5}$$

Proof. Write $c_r = x_r$. The difference equations are equivalent to $x_{r+4k} = c_r + D_r(k)$. These entries are all nonnegative precisely when $c_r \geq m_r$ for every r. The initial window additionally requires $\sum_r c_r = p_0$. Such nonnegative integer choices exist exactly when (4) holds: any surplus can be assigned to one c_r. The first window and the consecutive differences then give all the required windows, proving sufficiency as well as necessity.

For a feasible permutation with $p_k = 0$, the realization entries $x_k, \ldots, x_{k+3}$ are all zero and occupy one position in each residue class modulo four. The minimum realization entry in each class is therefore zero, which implies $c_r = m_r$. Summing the first four entries proves equality and gives (5). □

Computing the four minima requires $O(n)$ integer operations. This criterion and reconstruction from the zero position give two useful permutation membership checks; they are not practical replacements for aggregation over the full factorial candidate set.

3 Zero decomposition and weighted subset enumeration

3.1 Outward tails and complementary label sets

Definition 3.1. An *outward tail* with label word $(v_1, \ldots, v_m)$ starts with three zero entries. If the current last three entries are (a, b, c), its next label v appends

$$d = v - a - b - c \geq 0 \quad \text{and changes the last triple to } (b, c, d). \tag{6}$$

For a finite set S of positive integers, let $F(S)$ be the number of orderings of S that are valid outward tails, with $F(\varnothing) = 1$.

For a right tail at a zero window $p_k = 0$, the initial triple is $(x_{k+1}, x_{k+2}, x_{k+3}) = (0, 0, 0)$; the left tail uses the reversed triple specified in the proof below. Given the label word, every appended entry is forced, so $F(S)$ counts words rather than multiple realizations of one word. The definition uses the actual integer labels, not just their ranks.

Theorem 3.2 (Zero decomposition)**.** *For $n \geq 1$, with $U = \{1, \ldots, n-1\}$, one has*

$$a(n) = \sum_{S \subseteq U} F(S) F(U \setminus S). \tag{7}$$

Proof. If the zero of p is at position k, its right labels, read outwards, are $p_{k+1}, \ldots, p_{n-1}$. They form a tail starting from $(x_{k+1}, x_{k+2}, x_{k+3}) = (0, 0, 0)$. On the left, read the labels $p_{k-1}, \ldots, p_0$ outwards, and the realization entries in reverse order. This is another tail with initial triple $(x_{k+2}, x_{k+1}, x_k) = (0, 0, 0)$. The label sets of the two tails partition U.

Conversely, let ℓ and ρ be the label words of tails on S and $U \setminus S$. Place the entries appended by ℓ in reverse order to the left of four fixed zeros, and those appended by ρ in their generated order to the right. Both initial triples belong to this same four-zero block; no window of length four can reach both appended sides. The resulting window word is

$$(\mathrm{rev}(\ell),\, 0,\, \rho).$$

Every required window is contained in one of the two tail constructions or is the central zero window, so this is an admissible permutation. Its zero position is $|S|$ and recovers both tails uniquely. Hence the construction is a bijection and the sum counts each ordered permutation exactly once. The empty-tail case includes $n = 1$. □

In general $F(S)$ depends on the values in S. For example, $F(\{1,2,3,4,5\}) = 2$, with valid orders $(1,2,3,4,5)$ and $(1,2,3,5,4)$, whereas $F(\{1,2,3,4,7\}) = 1$. Indeed, the first four labels of any distinct-label tail must increase strictly; the fifth must be at least the fourth minus the first. These conditions give the stated examples directly. Thus replacing $F(S)$ by a function of $|S|$ would change the problem.

3.2 The weighted recurrence

Fix a maximum label N. Represent $S \subseteq \{1,\dots,N\}$ by a bit mask, and let $W(S,a,b,c)$ count the tails on S with last triple (a,b,c). Initialize $W(\varnothing,0,0,0) = 1$, with all other weights zero. For each unused $v \in \{1,\dots,N\} \setminus S$ with $v \geq a+b+c$, perform

$$W(S \cup \{v\}, b, c, v-a-b-c) \mathrel{+}= W(S,a,b,c). \tag{8}$$

Then

$$F(S) = \sum_{a,b,c \geq 0} W(S,a,b,c). \tag{9}$$

Weights, rather than Boolean reachability, are essential: different label orders may have identical mask and last triple.

Proposition 3.3 (Completeness of the recurrence)**.** *Equations* (8)–(9) *compute* $F(S)$ *exactly for every subset of* $\{1,\dots,N\}$ *and terminate after at most* N *transitions along any path.*

Proof. The empty tail gives the initial state. Every valid nonempty tail has a unique final label v; removing its final appended entry gives its unique predecessor tail and the transition (8). Conversely, every allowed transition appends a nonnegative integer and an unused label, so produces a valid tail. Induction on $|S|$ proves that its weight is the number of histories reaching that full state. Future extensions depend only on the used labels and the last three entries, so merging equal full states preserves these multiplicities. Each transition increases $|S|$ by one, proving termination. □

The triples lie in the finite set

$$\mathcal{T}_N = \{(a,b,c) \in \mathbb{Z}^3_{\geq 0} : a+b+c \leq N\}, \qquad |\mathcal{T}_N| = \binom{N+3}{3}. \tag{10}$$

This set is closed under (6): the next triple has sum $v - a \leq N$. Consequently the number of possible full states is at most $2^N \binom{N+3}{3}$; only reachable states need storage. A loose bound for transition attempts is $N2^N \binom{N+3}{3}$, rather than a factorial scan. This is an upper bound on work, not a prediction of measured running time.

These state and work bounds refer to the unpartitioned recurrence. The producer stores full mask–triple keys in a hash table with exact equality comparisons and integer weights, retaining the current and next layers alongside the F table. It processes disjoint prefix parts separately, so a full state may be processed in more than one part.

A prefix depth d means fixing the first d tail labels. For $d \leq 4$, these labels must increase strictly: the first four windows are cumulative sums of the first four appended nonnegative entries, and the labels are distinct. Conversely any strictly increasing word of at most four positive labels is realized by its first label and successive positive differences. Thus $F(S) = 1$ for $|S| \leq 4$, justifying the initialization of sets shorter than the chosen depth. Each longer tail belongs to exactly one fixed-prefix part. The reported run uses $d = 1$: nonempty tails are partitioned by their first label, and the empty tail is added once.

For any $S \subseteq \{1, \ldots, N\}$, a tail on S uses only labels in S, so its construction and $F(S)$ are unchanged by enlarging the ambient label universe. A table over $\{1, \ldots, N\}$ therefore supplies all coefficients needed in (7) for every $n \leq N + 1$.

For $n > 1$, the summands in (7) are paired by $S \leftrightarrow U \setminus S$, with no fixed subset. The implementation sums masks with their highest bit clear, one member of each pair, and doubles. This saves work; the answer still counts all ordered permutations.

3.3 Bounds on the producer's arithmetic

At layer h, each label order determines at most one tail, so every state weight and $F(S)$ with $|S| = h$ is at most $h!$. Even the sum over all states at that layer is at most $N!/(N-h)!$. Partial sums during a merge or a partition reduction are nonnegative subcounts of those exact totals. For $|U| = N$,

$$F(S)F(U \setminus S) \leq |S|!(N - |S|)! \leq N!.$$

The complete convolution is $a(n) \leq n!$ because it counts permutations. For $n > 1$, each partial sum of the half-convolution is at most $a(n)/2 \leq n!/2$, since its summands are nonnegative and the complementary pairing is exact. Doubling is therefore at most $n!$ as well. These bounds cover the intermediates, not just the observed final answer. For the reported run, $N = 21$ and $22! = 1124000727777607680000 < 2^{70}$. The producer uses checked unsigned 128-bit additions and multiplications, through the GCC integer extension [6]; no floating-point count arithmetic is used.

4 A second enumeration by inclusion–exclusion

The second method permits repeated positive labels and removes them by requiring full label coverage. Put $N = n - 1 \geq 1$ and $U = \{1, \ldots, N\}$. For $T \subseteq U$, let $L_h(T)$ count outward tails of length h whose labels all belong to T, with repetition allowed. Here $L_0(T) = 1$. Use the same triple transition (6), but retain no mask:

$$\begin{aligned} B_0^T(0,0,0) &= 1, \\ B_{h+1}^T(b, c, v - a - b - c) \mathrel{+}= B_h^T(a,b,c) &\quad (v \in T,\ v \geq a + b + c), \\ L_h(T) &= \sum_{(a,b,c) \in \mathcal{T}_N} B_h^T(a,b,c). \end{aligned} \tag{11}$$

Other initial weights are zero. Induction on h proves this recurrence counts exactly the repeated-label tails, since the final step and predecessor of a tail are unique. It is evaluated only for $0 \leq h \leq N$. Define

$$Q(T) = \sum_{k=0}^{N} L_k(T) L_{N-k}(T). \tag{12}$$

The index k is the position of the zero window. Left tails are again read outwards and then reversed to form the window word. Every other window has a positive label, so the zero is unique even though positive labels can repeat.

Theorem 4.1 (Inclusion–exclusion count)**.** *For $n \geq 2$,*

$$a(n) = \sum_{T \subseteq U} (-1)^{N - |T|} Q(T). \tag{13}$$

Proof. Fix a pair of tails of total length N contributing to (12); its left length fixes k. Let $V \subseteq U$ be the union of the labels they use. Its total coefficient in the right side of (13) is

$$\sum_{V \subseteq T \subseteq U} (-1)^{N-|T|} = \begin{cases} 1, & V = U, \\ 0, & V \neq U. \end{cases}$$

For $V \neq U$, this follows by expanding $(1-1)^{N-|V|}$; for $V = U$ there is one summand. This is Boolean-lattice Möbius inversion, whose inclusion–exclusion sign is described in [5, Section 3]. There are exactly N positive window positions. Covering all N positive labels therefore forces each to occur exactly once: a repetition would leave fewer than N distinct labels in those N positions. The surviving pairs are precisely those of Theorem 3.2, with coefficient one. □

The C++17 checker visits every one of the 2^N subsets, using dense arrays on $\mathcal{T}_N$. Unlike the producer, it uses neither distinct-label mask states within a walk nor complementary F tables, prefix partitions, hash-table aggregation, or a half-convolution. Its only input is n; the checker reads and compares no expected sequence count. Agreement with the native table is established externally from the completed output records. A direct implementation has an upper work bound $O(2^N N^2 \binom{N+3}{3})$, including all N layers and at most N label choices per state. The count arrays use $O(\binom{N+3}{3} + N)$ storage. The implementation also precomputes $O(N\binom{N+3}{3})$ transition indices and an $O(N^3)$ triple-index array, giving total auxiliary storage $O(N\binom{N+3}{3} + N^3)$.

4.1 Signed cancellation and intermediate bounds

Each tail step has at most N choices. For $0 \le h \le N$,

$$L_h(T) \le N^h, \qquad Q(T) \le (N+1)N^N. \tag{14}$$

Every state weight, layer partial total and product $L_k(T)L_{N-k}(T)$ is bounded by the corresponding nonnegative total above. In any order of subset processing, the absolute value of a signed partial sum in (13) is at most

$$2^N(N+1)N^N. \tag{15}$$

At $n = 22$, this is

$$269561447117208400366948292518477824 < 2^{118} < 2^{127}.$$

The bound increases with $N \ge 1$, so it covers all reported cases $2 \le n \le 22$. Unsigned 128-bit integers store nonnegative walk counts and products; a signed 128-bit integer stores the cancelling sum. The checker verifies layer bounds, per-subset bounds, subset coverage and the final interval $[0, n!]$. The case $n = 1$ is the single all-zero window and is counted as one. This arithmetic argument is the one applying to the checker used for the reported 22-term validation.

5 Structural consequences

Proposition 5.1 (Reversal and parity)**.** *For $n > 1$, $a(n)$ is even.*

Proof. Reversing a realization x reverses its window word, so reversal preserves $\mathcal{A}_n$. A fixed word under reversal would have $p_0 = p_{n-1}$, impossible for a permutation of length greater than one. Thus reversal pairs the admissible permutations without fixed points. □

The following observation was formulated after the numerical extension, during preparation for the manuscript. It is not used as a pruning rule in the enumeration.

Proposition 5.2 (Extension at either end)**.** *For every $n \geq 1$,*

$$a(n+1) \geq 2a(n). \tag{16}$$

Proof. Let $p \in \mathcal{A}_n$ have realization x. Set $s = x_n + x_{n+1} + x_{n+2}$. Its last old window is $x_{n-1} + s \leq n-1$, so $s \leq n-1$. Append the nonnegative entry $n - s$ to x. This preserves all old windows and creates the final window n, giving $(p, n) \in \mathcal{A}_{n+1}$.

Similarly let $t = x_0 + x_1 + x_2 \leq p_0 \leq n-1$. Prepending $n - t$ to x gives $(n, p) \in \mathcal{A}_{n+1}$. Each map is injective because deleting the added label recovers p. Their images are disjoint: the unique maximum n is at opposite ends, which are distinct positions even when $n = 1$. Together the maps prove (16). □

Corollary 5.3. *For every $n \geq 1$,*

$$2^{n-1} \leq a(n) \leq n!,$$

and $a(n)/2^{n-1}$ is nondecreasing.

Proof. The lower bound follows from $a(1) = 1$ and Proposition 5.2. The upper bound counts all permutations. Dividing (16) by 2^n gives the monotonicity. □

These are uniform mathematical statements. They do not determine an exact recurrence or an asymptotic growth rate from the finite table. Parity and the growth inequality are consistency checks, not independent exhaustive enumerations.

6 Computation and validation

6.1 Implementation and observed resources

The frozen producer is written in C17 with OpenMP and the GCC 128-bit extension. The completed run used maximum label 21 ($n = 22$), one worker, and prefix depth one. It computes the whole prefix $1 \leq n \leq 22$ by reusing the tail table for the smaller label sets. The source, build recipe, exact run records and validator records are provided with the computational package [7].

The recorded host was an AMD Ryzen 9 7940HS with eight cores and sixteen hardware threads, running Linux Mint 22.3 on x86-64, with recorded kernel 6.17.0-42-generic. The producer was built with GCC 13.3.0, C17, `-O3`, `-march=znver4`, `-mtune=znver4`, link-time optimization and `-fopenmp`. Its build also enables loop unrolling, frame-pointer omission and warning checks, and links the system cryptographic library for checkpoint integrity. The C++17 checker used GCC 13.3.0 with `-O3` and the same architecture tuning, without OpenMP. The unrestricted-window checker used Python 3.12.3 and arbitrary-precision integers. Each reported count was performed by a single worker or serial process; the processor's core count is not a claim of parallel execution.

The native run took about 297 seconds wall time, processing 1177119126 state occurrences and 2911882132 transitions across its prefix parts. Its peak tracked allocations were 3486204032 bytes, approximately 3.25 GiB; this is not a measured process RSS. The inclusion–exclusion count at 22 took about 251 seconds and had a measured peak RSS of 16404 KiB, approximately 16 MiB. Its dense triple state set has $\binom{24}{3} = 2024$ elements. The unrestricted Python count at 17 took about 102 seconds and had a measured peak RSS of 4334392 KiB, approximately 4.13 GiB. These are observations for the stated configurations, not projections to larger indices.

6.2 Different checks and their coverage

Table 1 records the complete domains actually checked. For the direct Python method, start with every nonnegative triple (a, b, c) satisfying $a + b + c \leq n - 1$, each of weight one, and an empty used-label mask. Apply (6) for unused labels $v \in \{0, \ldots, n-1\}$ for exactly n steps. Every witness is included: its first triple has sum at most its first window, and its successive windows give an allowed path. Conversely every complete path is a nonnegative witness with all n labels used. Lemma 2.1 then makes the path count a permutation count. This method does not generate by a zero separator or combine complementary tails.

The complete Python results at $1, \ldots, 12$ and 16 were obtained with earlier inline implementations of this recurrence, retained with their original commands and outputs. The retained parameterized program was qualified at $1, 2, 6$ and produced the complete result at 17. The combined row below describes the method's recorded coverage across these implementations, not execution of one frozen file at every listed index.

Method	Complete indices	Role or comparison
C weighted subset enumeration	$1, \ldots, 22$	Native complete counts
C++ repeated-label walks and inclusion–exclusion	$1, \ldots, 22$	Full recount by a distinct recurrence and implementation
Python unrestricted initial triples	$1, \ldots, 12, 16, 17$	Distinct generation without zero decomposition, using arbitrary-precision counts
All permutations: residue test and reconstruction from zero	$1, \ldots, 9$	Exhaustive comparison of membership predicates and totals

Table 1: Recorded complete domains. Comparisons with the native counts agree. The Python row combines the retained inline and parameterized implementations and does not include $13, 14, 15$ or any index above 17.

Before checking the six additional rows, inclusion–exclusion reproduced the historical $1, \ldots, 16$ prefix. Each run at $17, \ldots, 22$ then visited all 2^{n-1} subsets, including 2097152 subsets at 22, and matched the corresponding native result. The first additional value, at 17, was also fully recomputed by the unrestricted Python method.

The producer computes its values before an output check compares indices $1, \ldots, 16$ against an embedded historical table and rejects completion on disagreement. That check supplies no count to the recurrence and contains no expected value for $17, \ldots, 22$. The IE checker has no expected-count table; its results are compared with the native values externally.

Earlier Python tail-DP tests reproduced the known prefixes with maximum index $12, 13, 14, 15, 16$. At maximum index 16, the complete table of 32768 coefficients $F(S)$ agreed across prefix depths $0, 1, 2, 3, 4$. Those tests support implementation consistency; they are not additional independent mathematical models.

The producer and the inclusion–exclusion checker share the zero-anchor theorem, but their aggregation and repeated-label cancellation are different. The unrestricted method checks that shared generation assumption on its stated complete domain. C and C++ also share a compiler family; arbitrary precision in Python supplies a different arithmetic check at the reported indices. All local validation belongs to the same OpenAI/Codex maintenance context. Here "independent" describes the indicated algorithmic or implementation separation, not a third-party replication of the computations.

Original runnable checkers, inputs and completion records are included in the computational

package [7]. File-identity and result-consistency checks reuse that evidence on unchanged code; they do not constitute fresh enumerations. Incomplete runs, interrupted processes and partial state counts contribute no sequence term.

7 Results

Theorem 7.1 (Computational result)**.** *The complete enumerations described in Sections 3 and 4 agree for every* $1 \le n \le 22$ *and give the values in Table 2. In particular,*

$$a(22) = 2055385215352.$$

Computational proof. The masked recurrence is exhaustive by Proposition 3.3, and Theorem 3.2 converts its tail counts to the required ordered permutation counts. The completed native run produced all 22 rows. The second enumeration uses the exhaustive walk recurrence (11) and Theorem 4.1, with complete subset coverage at each index. The retained completion records and exact result comparisons agree row for row. The bounds in Sections 3.3 and 4.1 cover the fixed-width intermediates. The additional checks and their limits are those in Table 1. This is a finite computational result supported by the recorded executions, not a closed formula beyond their range. □

n	$a(n)$	n	$a(n)$
1	1	12	67646
2	2	13	301210
3	4	14	1397898
4	8	15	6786792
5	16	16	35154332
6	34	17	193009094
7	82	18	1125920768
8	252	19	6895579278
9	890	20	44276337188
10	3644	21	295719612912
11	15390	22	2055385215352

Table 2: The validated contiguous prefix. Rows 1–16 reproduce the historical OEIS baseline attributed to Hardin [1]; rows 17–22 are the six locally added counts, each fully recounted by inclusion–exclusion.

The canonical local b-file contains these 22 indexed values and preserves the historical prefix. Every row satisfies the proved parity and growth constraints. The first index without a complete validated value in this study is 23. This endpoint describes the completed and validated computations, rather than a mathematical limit of the methods. Neither those constraints nor the tail identities evaluate it from the existing table without further enumeration. No count at 23 or beyond is asserted here.

8 Conclusion

The zero label converts an inverse nonnegative-window problem into an exact pairing of tails on complementary positive label sets. It both removes realization multiplicity and permits finite-state

aggregation without scanning all permutations. Masked distinct-label enumeration and repeated-label inclusion–exclusion give matching complete counts through 22, extending the study's historical baseline by six values. The residue-minimum criterion decides realizability directly, while reversal and the two end-extension injections give evenness and a uniform exponential lower bound. The finite counts do not determine an asymptotic law; the useful outcome is an explicitly proved counting framework and a validated contiguous numerical prefix.

Data availability

The producer, build recipe, full count table, validation programs, original inputs and execution records are available in the GitHub repository and the Zenodo release series [7]. The manuscript source and its matching PDF are included in the computational package. The cited Zenodo DOI identifies the release series; individual deposited releases have distinct version-specific DOIs.

AI use disclosure

Large language models were used as research, code-review and drafting assistants during the A180207 workflow. Tool and vendor names below follow the supplied records; exact model versions and backends were not consistently authenticated.

Mathematical analysis. OpenAI/Codex assisted with the window-difference analysis, zero decomposition, feasibility and arithmetic proofs, and the later end-extension observation. NotebookLM supported consultation of prior methodological notes during strategy preparation; the sequence-specific identities were derived and checked locally. The mathematical claims are supported by the explicit arguments, primary sources and retained computational artifacts, not by conversational assertions or agreement among language models.

Algorithm and code. OpenAI/Codex assisted with producer development, maintenance, validation design and local evidence checks. The author performed the completed native run. The IE and Python validation processes were executed locally through OpenAI/Codex under the author's authorized validation scope. Received code-review reports identified their tools as ChatGPT, Claude, DeepSeek, GLM, Grok, Meta, MiniMax and Qwen; further reports came from Claude, Meta and MiniMax for follow-up code checks. These names describe the tool or vendor designations in the supplied reports; backend identities and reported remote executions were not independently authenticated. Exact model versions were not consistently verified. Review opinions are distinct from the local execution evidence, the complete inclusion–exclusion recounts and the stated Python checks.

Manuscript. OpenAI/Codex assisted with drafting, source checking and internal review of this manuscript. Numerical and validation claims were checked against the b-file, native result table, original completion records and validator sources. This internal work is not external manuscript peer review. Manuscript-review reports supplied by the author bore the designations ChatGPT, Claude, DeepSeek, Gemini, GLM, Grok, Meta, MiniMax and Qwen. These reports were evaluated critically; their designations do not authenticate distinct model families, and reported external computations were not added to the verified local validation domains without executable evidence. They are separate from the internal drafting checks.

The author, Carlo Corti, takes full scientific responsibility for the mathematical statements, algorithm, implementation, numerical results, validation boundaries and conclusions of this paper.

References

[1] R. H. Hardin, *A180207: Number of permutations of* $0, \ldots, n-1$ *representable as consecutive sums of 4 adjacent elements of a sequence of* $n+3$ *nonnegative integers*, The On-Line Encyclopedia of Integer Sequences, OEIS author date 15 August 2010. https://oeis.org/A180207. Historical 16-term entry snapshot used in this study; consulted November 2026.

[2] The OEIS Foundation Inc., *Consecutive-sum window-length variants*, entries A180205, A180206, A180208, A180209 and A180210, The On-Line Encyclopedia of Integer Sequences. Definitions listed with the A180207 entry; consulted November 2026. https://oeis.org/A180205, https://oeis.org/A180206, https://oeis.org/A180208, https://oeis.org/A180209, https://oeis.org/A180210.

[3] J. Konieczny, *On consecutive sums in permutations*, arXiv:1504.07156v5, 27 August 2021. Preprint DOI: 10.48550/arXiv.1504.07156. https://arxiv.org/abs/1504.07156v5.

[4] A. Higashitani and K. Kurimoto, *Permutations with small maximal* k*-consecutive sums*, arXiv:1801.00416v4, 27 May 2019. Preprint DOI: 10.48550/arXiv.1801.00416. https://arxiv.org/abs/1801.00416v4.

[5] G.-C. Rota, On the foundations of combinatorial theory I. Theory of Möbius functions, *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete* **2** (1964), 340–368. doi:10.1007/BF00531932.

[6] Free Software Foundation, *Using the GNU Compiler Collection (GCC)*, subsection "128-bit integers." https://gcc.gnu.org/onlinedocs/gcc/_005f_005fint128.html. Accessed November 2026.

[7] C. Corti, *Exact Enumeration of Four-Window Permutations: OEIS A180207 through* $n = 22$, GitHub repository and Zenodo archive, 2026. https://github.com/carcorti/A180207; https://doi.org/10.5281/zenodo.24194032. The Zenodo DOI identifies the release series.